

Lab 3 - Architectural Justification for Peer-to-Peer Carpooling and Expense Splitter System

Name: E Bhuvan
SRN: PES1UG24CS568
Date: 16/09/26

Architecture Selection:

I chose Microservices Architecture for my peer-to-peer carpooling and expense splitter system. Each service handles one clear job. It is designed to serve a large number of low cost users who send small requests through the day.

I would like to choose Microservices over others when I need to scale only the busy part and I need good failure separation and my loads are very different across services. This way I won't have a single point of failure.

Reason 1 - Independent scaling for route matching

Route matching demand spikes in morning and evening. Other services like Rate Trip and Notifications see steady low load. Since route matching usually sends tiny search requests, a small tuned service is normally sufficient.

This is where Microservices helps. If it detects 10000 users at peak time, it will start more copies of only Route Matching Service. It still meets NFR-002 which needs 2 second response. A layered or client-server design would force the full code to scale together and waste machines.

Reason 2 - Fault isolation across safety-critical and convenience features

Our system mixes safety work and comfort work. Licence check and payment are safety work. Ratings and alerts are comfort work. Microservices keeps them separate so one crash does not stop the rest.

This is where fault separation helps. If it detects that Notification Service is down, it will still allow Booking and Payment to run. Riders who are mid trip are not affected. A single block client-server design risks one bad module stopping the full platform.

Security Advantage

Sensitive data has a smaller damage area in Microservices. Licence papers stay only in User Service.

Payment details stay only in Payment Service. Each service uses its own store and its own access rules.

A bug in Notification Service cannot leak licence or payment data. That service never has access to those stores. This supports NFR-001 which needs encrypted and separate storage for ratings and papers and travel history.

Performance Benefit

Each service runs and scales on its own. Route Matching Service can run on machines tuned for fast map lookups. It is not slowed by heavy work like payment settlement or alert sending.

It sends short replies and stays quick for the rider. Other parts can stay busy or get updated without blocking search. This keeps the rider facing search fast even at peak load.

Trade-off Acknowledged

Microservices needs more care between services. Booking Service calls Route Matching and Payment over the network. This adds some delay and needs handling for partial faults.

However, this extra care is a fair price for scale and safety. Search stays fast at peak hours and payments stay safe in their own store. Therefore we accept this cost for our carpooling system.